

CS454, Assignment 1

Joshua Hernandez

September 2026

1 Libraries Used

This web crawler relies on several Python libraries to handle different parts of the scraping process. The Requests library handles the HTTP communication for the web crawler, allowing us to send requests to a selected URL and retrieve the content returned by the server. BeautifulSoup is then used to parse the HTML and extract the information we are looking for. Once the data has been collected, Pandas is used to organize it and save the crawler's output into a CSV file.

2 Process of Development

The development process of the web crawler can be broken down into three main parts: GET, PARSE, and STORE. Each part represents a different stage of retrieving, processing, and saving data from a webpage.

2.1 GET

For the GET process, the crawler uses the Requests library to handle communication with the selected website. Requests allows us to send an HTTP GET request to a URL and receive a response containing the content returned by the server. Once a successful response is received, the crawler is able to access the raw HTML content of the webpage, which can then be passed to the parsing stage.

With our GET process, it is very important that we filter our URLs correctly. We always want to make sure that the addresses being passed into the crawler are valid HTTPS URLs and valid URLs in general, especially when redirects are involved, since we do not always know what type of URL may come in.

- One of the major bugs I ran into was the crawler being redirected to a 'mailto:' address, such as 'mailto:sales@example.com', which caused the process to crash because it was not a valid webpage URL. Because of this, making sure that every URL starts with 'https://' became one of the main parts of the GET process.

For redirects, the crawler follows valid links found on each webpage while keeping track of URLs that have already been visited. Before following a link, it checks that the URL is valid so unsupported links do not cause the crawler to fail.

2.2 PARSE

For our parsing process, we use BeautifulSoup (BS4) together with the lxml parser. We chose lxml instead of the traditional html.parser because it is generally faster and gives us a stronger option as the crawler grows and begins handling more webpages.

BeautifulSoup reads through the raw HTML returned from our GET request and organizes it into a structure that we can work with. In our crawler, we pull the text from the main body of the webpage, which contains most of the visible page content. This allows us to remove unnecessary HTML formatting and keep the useful text before passing it into the storage process.

- One issue I ran into during parsing was the amount of empty space being extracted from the webpage. This made the data difficult to read once it was stored in the CSV file. To fix this, we strip the unnecessary whitespace from the extracted text so the final output is cleaner and easier to read.

2.3 STORE

For the storage process, I decided to use the Pandas library. I had never used Pandas before, but after doing some research, it seemed fairly simple to set up and use for organizing the scraped information. I created a fill table function that takes the collected data, organizes it into a table, and prepares it to be saved as a CSV file. This helped keep the storage portion of the crawler organized and separated from the scraping and parsing logic.

- My table is currently set up to append new information instead of overwriting the existing data. Because of this, if the crawler is run again from the beginning, it is important to use a separate CSV file for each URL run or sample if the results need to remain separate.

3 Results and Reflection

3.1 Seeds & Samples

- <https://fotmob.com> $\approx$ 1 hour 20 minutes
- <https://web-scraping.dev> $\approx$ 45 minutes
- <https://www.wsu.edu> $\approx$ 1 hour
- <https://www.uw.edu> $\approx$ 1 hour
- <https://www.scrapethissite.com> $\approx$ 1 hour 30 minutes

URL	Text
https://www.fotmob.com/players/1015128/elias-achouri	Elias Achouri San Diego FC Follow Height 99 Shirt 27 years Feb 10, 1999 Right Preferred foot Tunisia Country €1.8M Transfer value Jun 30, 2029 Contract end Position Primary Left Midfielder Others Right Winger, Left Winger LM RW LW Player traits Stats..
https://www.fotmob.com/players/1339287/ousseni-bouda	Ousseni Bouda San Jose Earthquakes Follow Height 7 Shirt 26 years Apr 28, 2000 Right Preferred foot Burkina Faso Country €917.4K Transfer value Dec 31, 2027 Contract end Position Primary Right Winger Others Left Midfielder, Left Winger, Striker LM RW LW ST Player traits Stats..
https://www.fotmob.com/players/794408/eduard-lowen	Eduard Löwen San Jose Earthquakes Follow Completed a transfer from St. Louis City for €365K Completed a transfer From St. Louis City €365K All transfers Height 23 Shirt 29 years Jan 28, 1997 Right Preferred foot Germany Country €1.6M..

Table 1: FotMob Sample.

3.2 Project Expansion

One way this project could be expanded is by improving the overall speed of the scraping process. I believe the GET portion could potentially be improved by using multithreading, allowing multiple webpages to be requested at the same time instead of processing each one individually. Another possible expansion would be handling the network connection at a lower level instead of relying entirely on the Requests library. This could involve creating and managing the connections ourselves using sockets. I am not sure how much of a performance improvement this would provide, since Requests already handles much of this efficiently, but it would be an interesting way to better understand and experiment with the networking side of the crawler.

3.3 Challenges and Takeaways

I believe the most difficult part of this project was implementing the adjacency matrix. The main challenge was keeping track of each discovered URL, assigning it a position in the matrix, and recording the connections between pages correctly. Although the idea sounded simple on paper, managing both the URL relationships and the visited page tracking became more challenging once I started implementing the logic in code.

Overall, this project was very enjoyable for me because I was able to work with several libraries that I had wanted to gain experience with. In particular, I became more comfortable using Pandas and Requests, which I feel will be valuable for future projects.